

Computer Security 2026

Side channels defenses and Spectre (HW security)

Roberto Guanciale

Countermeasures

Timing attack

Access driven attack

Trace driven attack

Eliminating the side channel

— — —

- Totally disable caches
- Disable caches for the victim
 - For trace driven and access driven attacks
- Disable caches for the attacker
 - To prevent him to prime/probe
- Huge performance impact

Cache flushing

- Only for access driven attacks
 - Flush at the beginning
 - Flush at exiting
- If attacker shares memory, then flush at beginning and exiting
 - Attacker could prime the cache and measure execution time of the victim
 - Attacker could measure if something is in the cache after the execution of the victim

Cache partitioning (colouring)

- For access driven and trace driven attacks
- Usually via MMU.
- Assume 8 KB cache, 4KB pages, 2 processes
 - Even pages are reserved for process 1
 - i.e. 100, 8K + 100, 2M + 16K + 100
 - Odd pages are reserved for process 2
 - I.e. 4K + 100, 10K + 100, 2M + 18K + 100
- Attacker cannot monitor cache accesses
- Problems
 - Under-utilization of caches
 - Difficult for several processes

Actively eliminating the side channel

- For access driven attacks
 - Victim can explicitly evicts all lines that can leak secrets (i.e. SBox) when exiting
 - Victim can normalize the cache by priming it with data that can be accessed
-
- For timing channels, victim can wait until its Worst Case Execution Time

Reducing side-channel bandwidth

— — —

- Introduce noise
 - Trace-driven and access driven: by accessing random data
 - Timing-channels: by adding random computations
- Reduce precision in measuring time
 - E.g. introduce random noise
 - (Attacker could access an external source for timing, like GPS signal or network protocols)

Randomization

— — —

- Randomize position of data structure
 - e.g. $\&M' = \&M + r$
- Randomize layout of data structure
 - e.g. $M[i] = M[\pi(i)]$
- Randomize code
 - e.g. Change order of bytes in encryption

$$C[1] = M[P[1]] + K[1]$$

$$C[0] = M[P[0]] + K[0]$$

Detection

— — —

- Identify a process that primes caches:
 - At context switch cache contains suspicious (unrelated) addresses
- Identify a process that probes caches:
 - Frequent syscall requests to access system time

Constant time (power consumption, etc) programming

- Cache state depends on sequence of memory accesses (loads, stores, instruction fetches) performed
- Execution time depends on sequence of memory accesses and sequence of instruction executed
- Make these constants
- Constant time programming
 - Never branch on secret
 - Ensure that memory accesses are secret-independent
 - Usually too restrictive

Modular exponentiation (i.e. RSA)

```
function modular_pow(base, exponent, modulus)
  if modulus = 1 then return 0
  result := 1
  base := base mod modulus
  while exponent > 0
    if (exponent mod 2 == 1):
      result := (result * base) mod modulus

    V = (exponent mod 2) * base + 1 - (exponent mod 2)
    result := (result * V) mod modulus
    exponent := exponent >> 1
    base := (base * base) mod modulus
  return result
```

Other forms of side channels

Browser fingerprinting: what & why?

— — —

- Passive/active collection of browser + device characteristics to create a unique identifier (a “fingerprint”).
- User-Agent, screen size & color depth, timezone, installed fonts, list of plugins, audioContext features, platform, language.
- Enables cross-site tracking even when cookies are blocked or cleared
- In large studies (4M+ fingerprints), over 80% of collected browser fingerprints were unique in the dataset.
- Around 90% of attributes remain stable across months, meaning uniqueness persists across sessions.
- Recent studies (PETs Symposium 2025) show uniqueness can vary by age, income, and other demographics

How to reduce fingerprintability

— — —

- Choose privacy-focused browsers: Brave with anti-fingerprinting on.
- Block third-party scripts, disable unnecessary plugins
- Use extensions carefully
- Make your fingerprint common: Keep default settings, use common window sizes, don't install unique fonts or plugins
- Operational options: Use private VMs, containerised browsing, or always browse in a hardened profile for sensitive tasks.
- Legal/ethical note: For site owners, disclose tracking, get consent when required, and prefer privacy-preserving analytics.

Network analysis: Video identification

— — —

HTTP Adaptive Streaming (DASH):
Videos are split into segments of
variable size / bitrates.

The pattern of chunk sizes / timing
leaks info.

Bytes-Per-Second / Bits-Per-Peak
(BPS / BPP): Many works extract a
time series of bytes or peak
traffic bursts to fingerprint a
video.

CNNs or sequential models applied
to traffic feature sequences to
classify video titles.

Recent work shows that even in LTE
networks, with sniffing of radio
signals, adversaries can identify
which video is being watched with
high accuracy (up to ~ 0.985).

Network analysis: SSH — Password extraction

— — —

SSH sends small packets for interactive typing; measuring packet arrival times reveals the time between keystrokes.

Statistical models can turn timings into likely key sequences and recover passwords.

Protocol/implementation details may leak exact password length.

server/client can buffer keystroke packets

Traffic shaping

Spectre



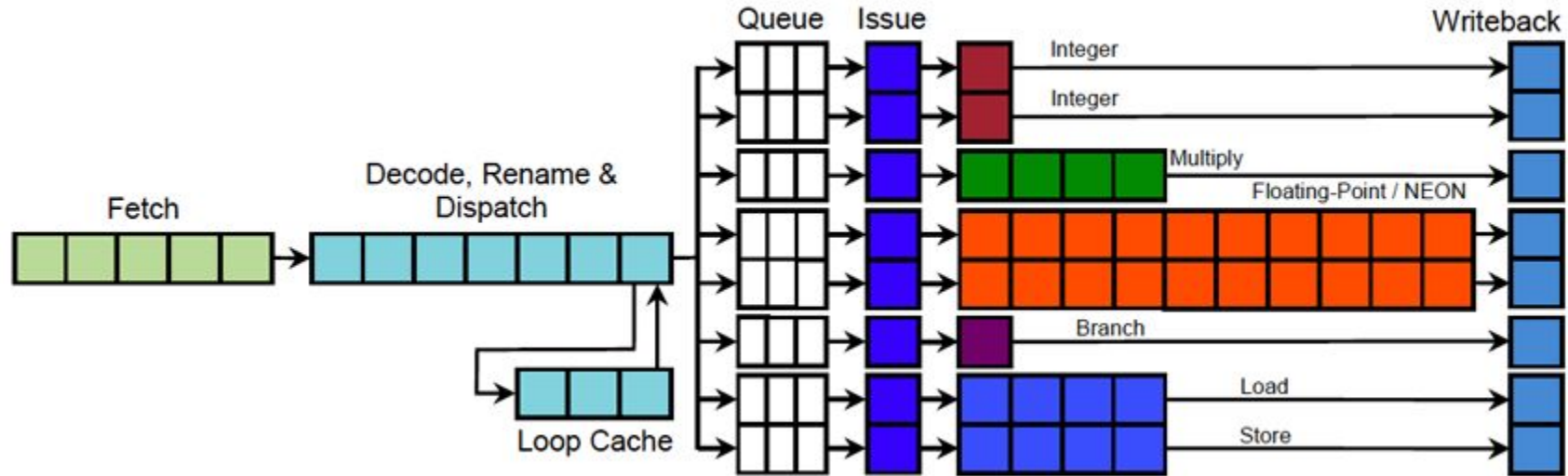
Spectre

```
int F(R0) {  
    R1 = LOAD #size-A  
    if 0 <= R0 < R1 {  
        R2 = LOAD #A[R0]  
        R3 = LOAD #B[R2]  
        STORE #output R3  
    }  
}
```

// ASSUME #A is public, #B is public and
// every i. $0 \leq A[i] < \text{size-B}$
Is this program non-interferent?
Is this program constant time?

Relative old pipeline ARM A57

— — —

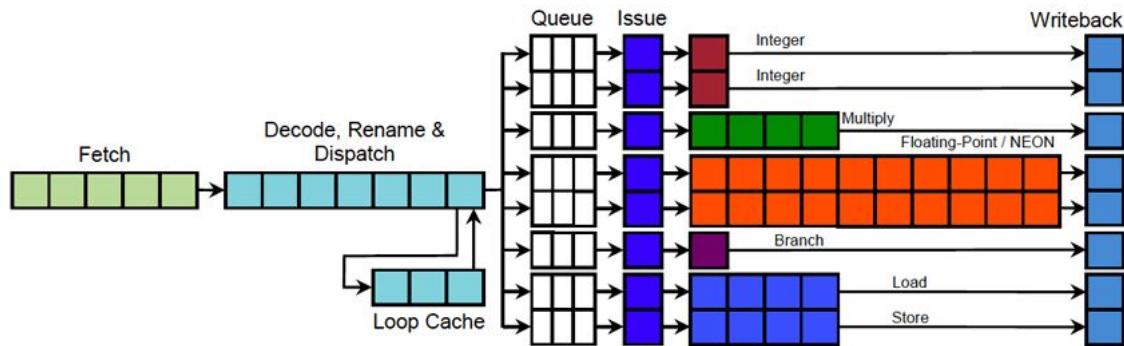


Out of order execution

```
R1 = LOAD[R0]
```

$$R2 = R3 + 3$$
$$R4 = R2$$
$$R5 = R4 + R1$$

- Memory accesses can be really slow:
 - Execute out-of order when possible



Speculative execution

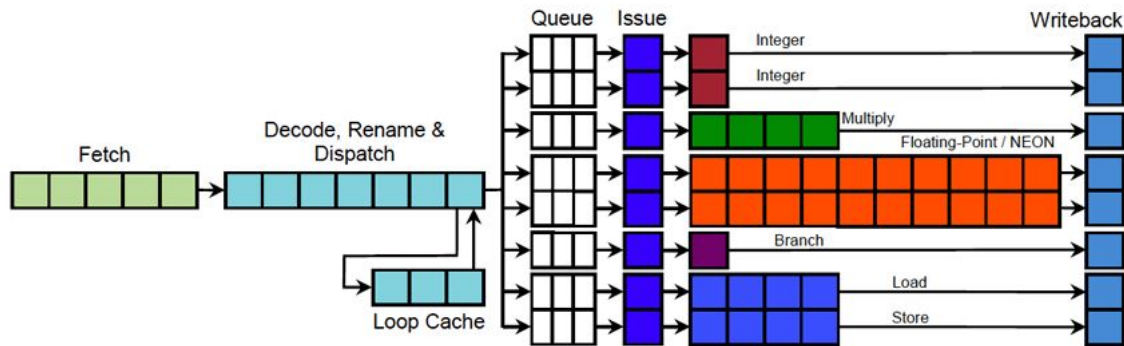
R1 = 0

loop: R2 = R2 * R1

R1 = R1 + 1

if R1 < 100 jmp loop

call printf R2



- Pipeline flushes are expensive
 - Guess $R1 < 100$
- Speculate executions, except memory store

Spectre

```
int F(R0) {                                // R0 = #key - #A
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {                      // false
        R2 = LOAD #A[R0]                 // #A+R0=#A+#key-#A    R2=key
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

```
// ASSUME #A is public, #B is public and
// every i. 0 <= A[i] < size-B
```

Spectre

```
int F(R0) {                                     // R0 = #key - #A
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {                           // false
        R2 = LOAD #A[R0]                       // #A+R0=#A+#key-#A    R2=key
        R3 = LOAD #B[R2]                       // Cache line accessed depends on
        STORE #output R3                      R2 (key)
    }
}
```

```
// ASSUME #A is public, #B is public and
// every i. 0 <= A[i] < size-B
```

Spectre v1 attacks

- Two gadgets:
 - Reading gadgets (i.e., buffer overread)
 - Transmitting gadget (i.e., access to memory)
- Attacker poisons the branch predictor
- Attacker uses standard cache driven attack (i.e., prime/probe via access driven attack)

Spectre v1 defence: instruction serialization

```
int F(R0) {  
    R1 = LOAD #size-A  
    if 0 <= R0 < R1 {  
        lfence  
        R2 = LOAD #A[R0]  
        R3 = LOAD #B[R2]  
        STORE #output R3  
    }  
}
```

Spectre v1 defence: instruction serialization

```
int F(R0) {  
    R1 = LOAD #size-A  
    if 0 <= R0 < R1 {  
        R2 = LOAD #A[R0]  
        lfence?  
        R3 = LOAD #B[R2]  
        STORE #output R3  
    }  
}
```

Spectre v1 defence: index masking (SLH)

— — —

```
int F(R0) {  
    R1 = LOAD #size-A  
  
    if 0 <= R0 < R1 {  
        // Create a boolean mask  
        R4 = (R0 >= 0) & (R0 < R1)  
        R0 = R0 & R4  
  
        // (R0 will be 0 if out of bounds)  
        R2 = LOAD #A[R0]  
  
        R5 = (R2 >= 0) & (R2 < size-B)  
        R2 = R2 & R5  
  
        R3 = LOAD #B[R2]  
        STORE #output R3  
    }  
}
```

Spectre v1 defence: hardware based

— — —

- Branch Target Injection Mitigations
- Array Indexing Bound Checks in Hardware
- Non speculative memory load
- Non speculative cache effects
- Taint tracking

Spectre v2

— — —

```
f:  R2 = LOAD #A[R2]
```

...

```
g:  R2 = LOAD #Key[R0]
```

```
    R3 = LOAD #A[R2]
```

...

```
R3 = &f
```

...

```
JMP R3
```

Spectre v2

```
f:  R2 = LOAD #A[R2]
```

```
...
```

```
g:  R2 = LOAD #Key[R0]  // reading gadget  
    R3 = LOAD #A[R2]    // transmitting gadgets
```

```
...
```

```
R3 = &f
```

```
...
```

```
JMP R3                // processor may guess target of jump
```

Spectre v2: hardware defenses

- Indirect Branch Restricted Speculation (IBRS):

A hardware control that prevents indirect branches from using speculative predictions unless explicitly allowed.

- Single Thread Indirect Branch Predictors (STIBP):

Prevents one thread from influencing the branch predictor state of another thread on the same core, protecting against cross-thread attacks.

Spectre v2: Retpoline (from google)

— — —

```
f:  R2 = LOAD #A[R0]
```

```
...
```

```
g:  R2 = LOAD #Key[R0]
```

```
    R3 = LOAD #A[R2]
```

```
...
```

```
trampoline adr:
```

```
    stack.ret_ptr = adr
```

```
    ret;
```

```
trampoline(r3);
```

```
while (true) {}
```

Spectre v4

STORE #A[0] key

R1 = 0

R2 = 0

STORE #A[R1] 0

R3 = LOAD #A[R2]

R4 = LOAD #B[R3]

// A[0] = 0

Spectre v4: Store Bypass

```
STORE #A[0] key
```

```
R1 = 0
```

```
R2 = 0
```

```
STORE #A[R1] 0
```

```
R3 = LOAD #A[R2]
```

```
R4 = LOAD #B[R3]
```

```
// A[0] = 0
```

```
// Processor may speculate R2 != R1
```

```
// and execute load before store
```

Spectre v4: Store Bypass

```
STORE #A[0] key
```

```
R1 = 0
```

```
R2 = 0
```

```
STORE #A[R1] 0 // A[0] = 0
```

```
lfence
```

```
R3 = LOAD #A[R2] // Processor may speculate R2 != R1  
// and execute load before store
```

```
R4 = LOAD #B[R3]
```

Spectre: straight line speculation

```
int F() {  
    STORE #A[0] = 0  
}
```

```
F();  
R2 = LOAD #A[0]           // Processor may execute in speculation  
R3 = LOAD #B[R2]
```

Spectre: phantom instructions

— — —

```
int F() {  
    R2 = LOAD #A[R1];  
    R3 = LOAD #B[R2];  
}
```

NOP;

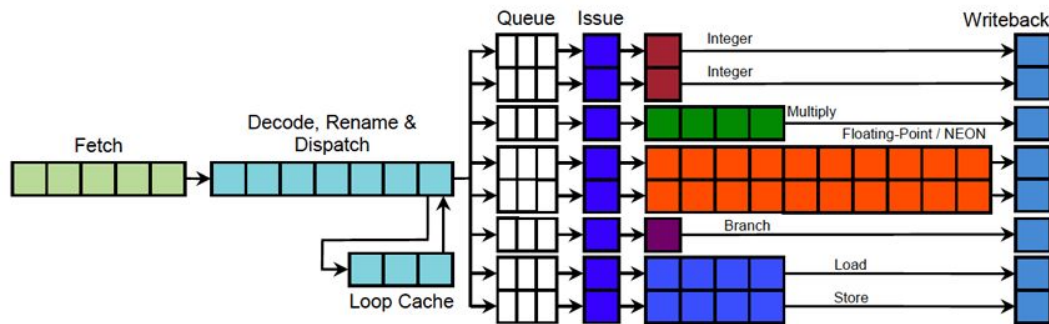
...

Spectre: phantom instructions

```
int F() {  
    R2 = LOAD #A[R1];  
    R3 = LOAD #B[R2];  
}
```

NOP;

...



Spectre: speculative prefetching (Apple M1)

```
for i : 0 to 100 {  
    x = i % 50  
    x = x & ..  
    y = A[x];           // after reading 0, 1, ... 49, processor  
                        // may speculate index 50  
    z = B[y];  
    ...  
}
```

Meltdown attack

— — —

Normal proceed for executing LOAD [R1]

1. Identify instruction to fetch
2. Fetch the instruction
3. Decode the instruction
4. Wait for flying instructions that are modifying R1
5. Read a = R1
6. Wait for flying instructions that are writing in a
7. Translate a from virtual to physical address pa
 - a. (read page tables)
8. Check if pa is readable
 - a. (read page tables)
9. Read pa
 - a. If pa is in cache => hit
 - b. Evict cache line and load pa.

Spectre Effects

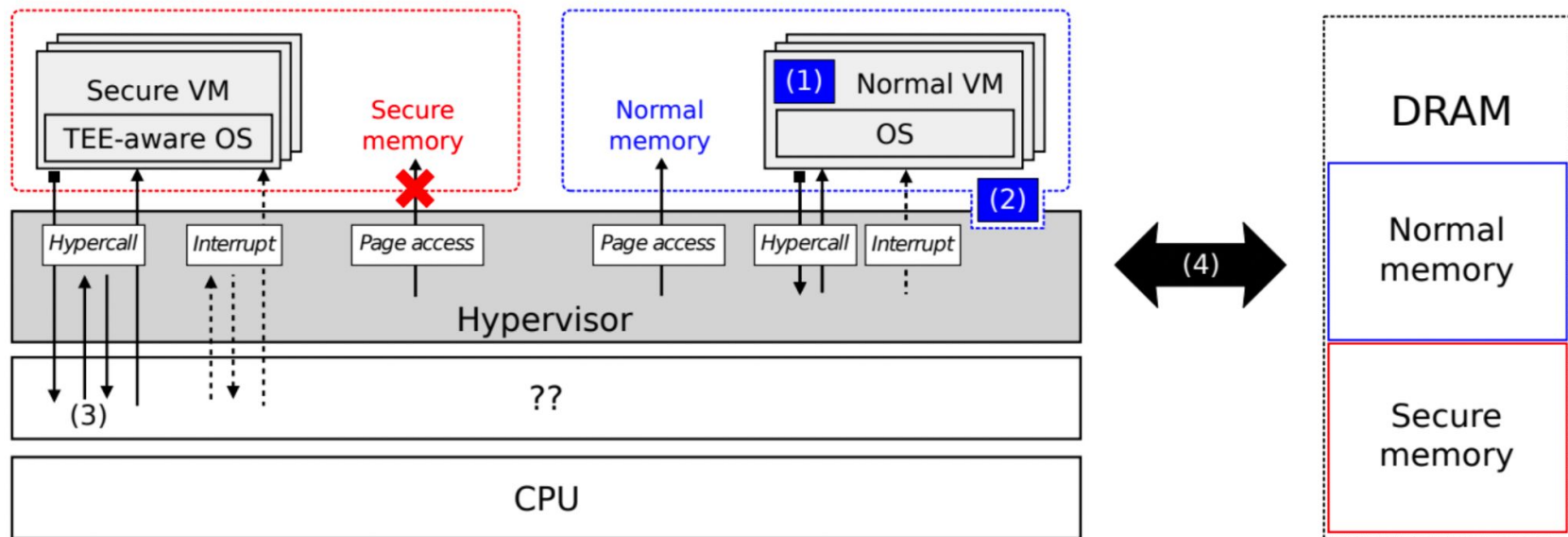
- Web Browsers
 - crafted JavaScript code, an attacker could exploit speculative execution to read memory from the browser, including sensitive data like passwords or session tokens across different domains.
 - Google Chrome introduced Site Isolation, a feature that isolates webpages from each other in different processes.
 - Mozilla Firefox and Microsoft Edge reduced the resolution of timers to make it harder to perform fine-grained side-channel attacks.
- Enclaves (see later)
 - The SGXPectre attack demonstrated that even the highly secure Intel SGX (Software Guard Extensions)
- Linux Kernel (Meltdown and Spectre):
 - allowing unprivileged user processes to access kernel memory. This vulnerability could expose sensitive kernel data such as passwords, encryption keys, and user information.
 - Retpoline

Isolation

— — —

Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV



Trusted Execution Environments

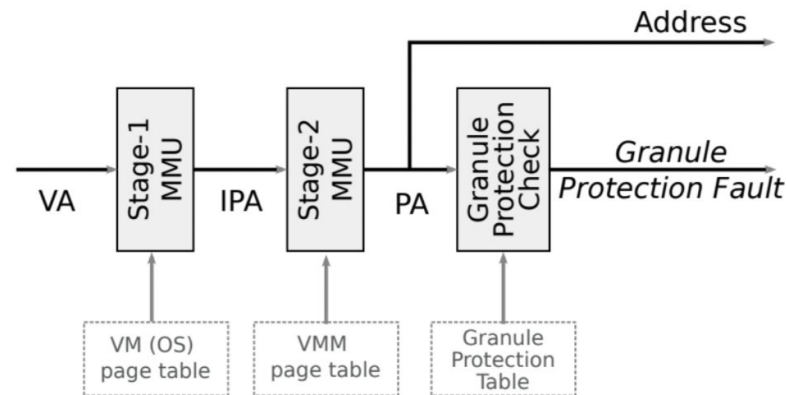
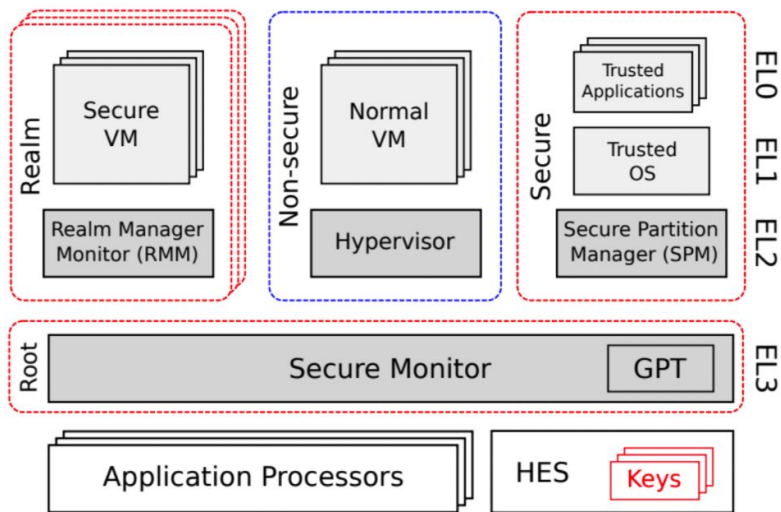
— — —

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV

- Do not trust hypervisor (and usually owner of the physical machine)
 - Also provide some sort of memory encryption to prevent physical attacks or attacks from devices
- Resource management delegated to hypervisor
 - (e.g., no protection against DOS)
- Provide some mechanism for remote attestation
 - Allow a trusted component to “measure” (e.g., digest of executing code and configuration)
 - Allow to establish secure channel via crypto
 - Allow to report measurements

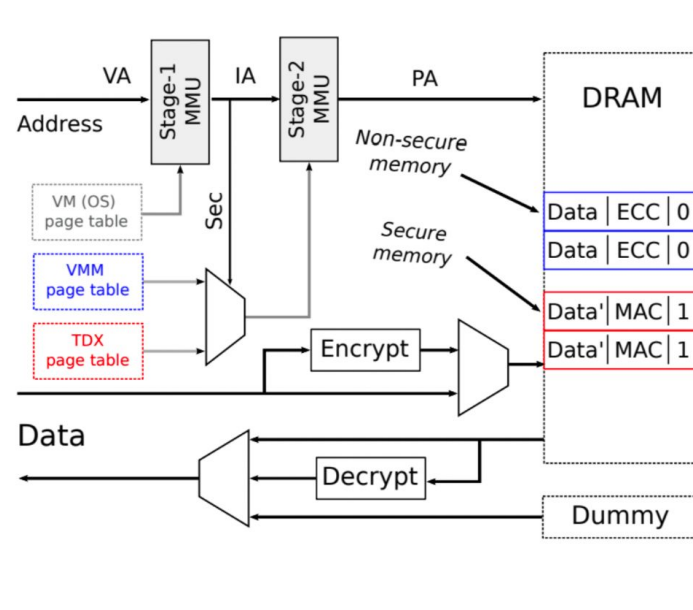
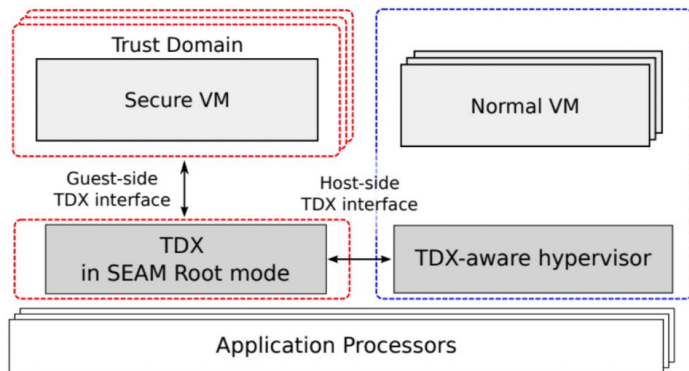
Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, **ARM CCA**, AMD SEV



Trusted Execution Environments

Intel SGX, ARM trustzone, **Intel TDX**, ARM CCA, AMD SEV



Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, **AMD SEV**

